

2026-05-05-p1-f05-hook-based-extensibility-design

P1-F05 — Hook-Based Extensibility — Design Spec

Date: 2026-05-05 **Author:** Claude Opus 4.7 (1M context) + user (milos85vasic.2nd@gmail.com)
Phase / Feature: Phase 1, Feature 5 of docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md **Status:** APPROVED in brainstorming, awaiting user review of written spec **Successor:** to be handed to superpowers:writing-plans for executable plan **Predecessor:** Feature 4 (Git Worktree Agent Isolation) — closed 2026-05-05 (commits 7ba8907...8548524)

1. Goals, non-goals, success criteria

1.1 What we’re building

A claude-code-style hook system that lets users extend HelixCode’s lifecycle behaviour by writing shell scripts. Users author `~/ .helixcode/hooks.yaml` (and optionally `<project>/ .helixcode/hooks.yaml`) listing event-type → shell-script mappings; HelixCode loads them at startup and fires them at documented runtime points.

The 9 lifecycle events:

Event	Fired at
before_tool_call	every tool execution, BEFORE the tool’s Execute runs
after_tool_call	every tool execution, AFTER (success or error)
before_edit	tool execution where toolName ∈ {Edit, Write, MultiEdit}, before
after_edit	same, after
before_bash	tool execution where toolName == Bash, before
after_bash	same, after
on_error	any tool error or LLM error in the agent’s message loop
on_compaction	a successful auto-compaction event (F01’s AutoCompactor)
on_plan_approval	F08 Plan Mode approval gate (F05 ships the stub; F08 wires production caller)

A hook handler that exits non-zero **blocks** the originating operation. Stdout JSON modify-payload is parsed into the event's `Data` map for downstream handlers (propagation back to the operation's params is deferred per N1).

1.2 Goals (priority order)

- **G1 — No bluff.** Per Constitution Article XI §11.9, every PASS in this feature carries positive runtime evidence. The Challenge proves a real `before_bash` hook with a real shell script blocks a real `rm -rf` and the marker file is preserved.
- **G2 — Extend, don't parallelise.** Same lesson as F01–F04. The existing `internal/hooks/` package is fully functional (Manager, Trigger* methods, Hook struct, Event struct, Executor with sync/async/wait). F05 adds 6 new HookType constants, a YAML loader, a generic shell-runner HookFunc, and a Blockers helper. The package's existing HookFunc(ctx, *Event) error signature already supports both block (non-nil error) and modify (mutate event.Data).
- **G3 — Wiring is the load-bearing piece.** The existing `internal/hooks/` package was unwired (zero Trigger calls in production code). F05's value comes from actually firing events at the documented lifecycle points: `internal/tools/registry.Execute` (6 of 9 events: BeforeToolCall, AfterToolCall, plus conditionally BeforeBash/AfterBash/BeforeEdit/AfterEdit), `internal/llm/compression/auto_compactor.go` (OnCompaction), `internal/agent/agent.go` (OnError + the OnPlanApproval stub via RequestPlanApproval).
- **G4 — Config-driven, not Go-plugin-based.** Users author shell scripts referenced from YAML; HelixCode exec's them with the event payload on stdin. Matches claude-code's actual user-facing hook system. No new dependency, no plugin. Open minefield.
- **G5 — Full surface.** 5 Cobra subcommands (`list`, `test`, `enable`, `disable`, `validate`) + `/hooks` slash command. Mirrors F02/F04 polish.

1.3 Non-goals (explicit out-of-scope for F05)

- **N1.** Modify-payload propagation. Scripts can write JSON to stdout that mutates the event's `Data` map for downstream handlers — but F05 does NOT yet propagate that modified data back into the originating operation's params. The reader path exists; the writer path defers.
- **N2.** Production caller for RequestPlanApproval. F05 ships the stub function (which dispatches `on_plan_approval`); F08 (Plan Mode) wires it into the actual plan-mode flow. F05 has no production caller in the agent message loop.
- **N3.** Hook UI / visualisation (a “what hooks fired and when” timeline). YAGNI.
- **N4.** Per-event payload schema validation. F05 documents schemas in code comments; runtime validation defers.
- **N5.** Worktree-aware hook discovery. Hooks always come from `~/helixcode + <repoRoot>/helixcode`, regardless of active worktree (from F04).
- **N6.** Built-in hooks shipped with HelixCode (e.g., a default audit hook). User's responsibility to author their own.
- **N7.** Hot-reload of `hooks.yaml`. Hooks are loaded once at CLI startup; changes require restart.

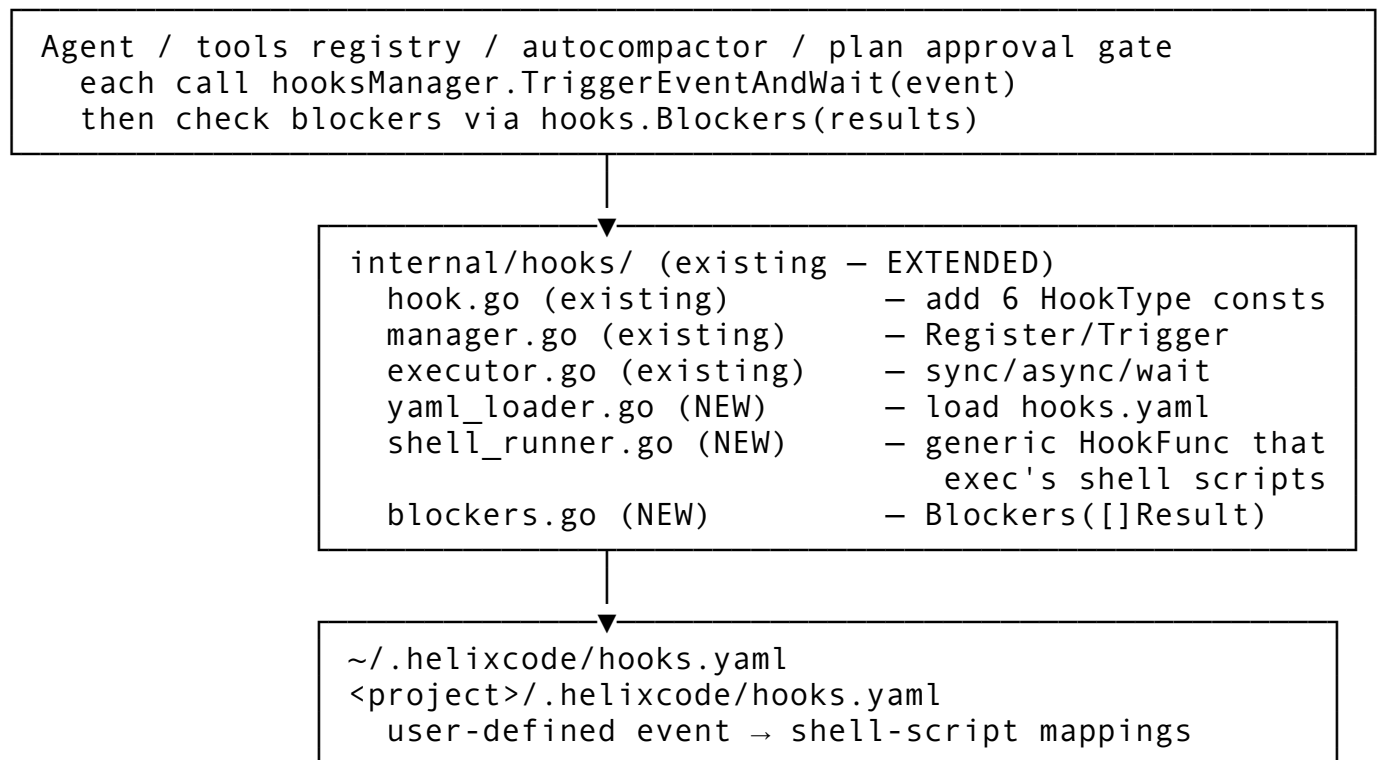
1.4 Success criteria

- **S1.** `make verify-compile` exits 0 with the new files + extended types + 5 wiring points.
- **S2.** Unit tests for `internal/hooks/yaml_loader.go`, `shell_runner.go`, `blockers.go`, and the 5 wiring points pass with `-race`. Most tests use REAL bash against `t.TempDir()` ephemeral scripts.

- **S3.** Integration test (`-tags=integration`, no mocks) demonstrates: a real `before_bash` shell script that exits 1 prevents the tool's `Execute` from running; an `after_tool_call` shell script that writes to a JSONL log captures all 3 of 3 tool calls.
- **S4.** Challenge under `tests/e2e/challenges/hooks/` runs 3 scenarios with runtime evidence pasted into the close-out commit body. Mutation test (commenting out the `Blockers` check in `tools.registry.Execute`) makes S1 fail.
- **S5.** Anti-bluff smoke (`grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/hooks/ ...`) returns clean.
- **S6.** All 5 Cobra subcommands work end-to-end: `helixcode hooks list + validate + test + enable + disable`. The `/hooks` slash command is discoverable via the registry.
- **S7.** YAML schema enforced: `apiVersion: helixcode.hooks/v1` mandatory; project-over-user precedence on identical `id`; disabled hooks loaded but not registered.

2. Architecture

2.1 Topology



User writes YAML at `~/.helixcode/hooks.yaml` listing event-type → shell-script. At startup, `cmd/cli/main.go` loads the YAML, wraps each entry in a `shellRunner HookFunc`, and calls `manager.Register`. The 5 wiring points dispatch events; if any registered handler returns non-nil error, `hooks.Blockers(results)` flags it as a block.

2.2 Why this shape

- **One existing package extended.** Mirrors F01–F04. The existing `internal/hooks/` package's API (`HookFunc`, `Manager.Trigger*`, `Hook/Event` structs) is sufficient — only adding event-type enum entries and supplementary wrappers.

- **Shell-script hooks**, not Go plugins. Matches claude-code’s user-facing model. Zero new dependencies. End users have no compile step.
- **Five wiring points, all using the existing hooks**. Manager from `session.Manager`. The session manager already has a `hooksManager *hooks.Manager` field (F02-era introduction). F05 reuses it; doesn’t introduce a parallel registry.

2.3 Component responsibilities

Component	Responsibility
<code>internal/hooks/hook.go</code> (extended)	6 new <code>HookType</code> constants: <code>HookTypeBeforeToolCall</code> , <code>HookTypeAfterToolCall</code> , <code>HookTypeBeforeBash</code> , <code>HookTypeAfterBash</code> , <code>HookTypeOnCompaction</code> , <code>HookTypeOnPlanApproval</code> . The existing <code>HookTypeBeforeEdit</code> , <code>HookTypeAfterEdit</code> , <code>HookTypeOnError</code> are reused.
<code>internal/hooks/yaml_loader.go</code> (NEW)	<code>FileLoader</code> struct (<code>UserPath</code> , <code>ProjectPath</code> fields). <code>Load(ctx) ([]*Hook, []string, error)</code> returns the registered hooks + source-path list. Validates <code>apiVersion: helixcode.hooks/v1</code> ; rejects unknown event types at load with a logged warning (other hooks continue). Project-file entries override user-file entries with the same id.
<code>internal/hooks/shell_runner.go</code> (NEW)	<code>NewShellRunner(scriptPath string, timeout time.Duration)</code> <code>HookFunc</code> returns a closure. The closure: serialises event payload to JSON, exec’s the script with <code>stdin</code> , applies the timeout via <code>exec.CommandContext</code> , captures <code>stdout+stderr</code> , parses <code>stdout</code> as JSON for the modify-payload (read-only in F05), returns error wrapping <code>stderr</code> on non-zero exit. Missing script at fire time → fail-closed (return error).
<code>internal/hooks/blockers.go</code> (NEW)	<code>Blockers(results []*ExecutionResult) []error</code> collects non-nil errors from sync results. Helper for callers to check “did any hook block?”
<code>internal/tools/registry.go</code> (extended)	<code>Execute</code> dispatches <code>BeforeToolCall</code> first; if blocked, return <code>fmt.Errorf("operation blocked by hook %s: %s", hookID, stderr)</code> . Then conditionally dispatches <code>BeforeBash</code> (if <code>toolName == "Bash"</code>) or <code>BeforeEdit</code> (if <code>toolName ∈ {Edit, Write, MultiEdit}</code>) — block here also aborts. After the tool runs (success or error), dispatches <code>AfterToolCall</code> + conditional <code>AfterBash/AfterEdit</code> . After-event blocks are LOGGED but do not undo the operation.
<code>internal/llm/compression/auto_compactor.go</code> (extended)	After a successful compaction, dispatch <code>OnCompaction</code> synchronously with payload <code>{before_size, after_size, messages_compacted}</code> . Block here aborts the next message-loop iteration (returns error from compaction).

Component	Responsibility
internal/agent/ agent.go (extended)	On any tool error or LLM error in the message loop, dispatch <code>OnError</code> with <code>{error_message, error_type}</code> <code>async</code> (fire-and-forget; the error already happened). Add <code>(c *agent) RequestPlanApproval(ctx, plan)</code> method that dispatches <code>OnPlanApproval</code> synchronously and returns the aggregated blocker — no production caller wires it in F05; F08 will.
cmd/cli/ hooks_cmd.go (NEW)	Cobra <code>helixcode hooks {list,test,enable,disable,validate}</code> group. <code>list</code> shows tabwriter <code>ID/EVENT/SCRIPT/PRIORITY/ENABLED/SOURCE</code> . <code>test <event></code> simulates event + runs handlers, prints results. <code>validate</code> parses YAML without side effects. <code>enable/disable</code> mutate the <code>enabled</code> flag in user's <code>hooks.yaml</code> in place.
internal/commands/ hooks_command.go (NEW)	<code>/hooks</code> slash command. Subactions: <code>bare/list</code> , <code>test <event></code> . Aliased to <code>/hk</code> .
internal/commands/ builtin/register.go (extended)	<code>RegisterBuiltinCommandsWithHooks(registry, hooksMgr)</code> error. Original <code>RegisterBuiltinCommands</code> signature unchanged.
cmd/cli/main.go (extended)	New <code>initHooks(ctx)</code> method that constructs a <code>FileLoader</code> , loads YAML, registers each enabled hook into the existing <code>session.Manager.hooksManager</code> . Called from <code>Run()</code> after <code>initWorktree(ctx)</code> .

3. Data shapes

3.1 YAML schema

`apiVersion: helixcode.hooks/v1`

`hooks:`

```

- id: audit-tool-calls           # required, unique within
  file scope
  event: before_tool_call        # required, must match a
  known HookType value
  script: ~/.helixcode/scripts/audit.sh  # required, ~
  expansion supported
  priority: 100                  # optional, default 0;
  higher runs first
  async: false                   # optional, default
  false
  timeout: 5s                    # optional, default 0 =
  no timeout (Go duration string)
  enabled: true                  # optional, default true
  description: "log every tool call to a JSONL audit file"
```

3.2 Event payload contract (script's stdin)

A JSON document is written to the script's stdin:

```
{
  "type": "before_tool_call",
  "timestamp": "2026-05-05T10:23:45Z",
  "session_id": "<uuid or empty>",
  "source": "tool_registry",
  "data": {
    "toolName": "Bash",
    "params": { "command": "rm -rf /tmp/x" }
  }
}
```

3.3 Modify-payload contract (script's stdout, OPTIONAL)

A script may print a JSON document to stdout:

```
{ "data": { "toolName": "Bash", "params": { "command": "echo
hi" } } }
```

shellRunner parses this and merges data into event.Data for downstream handlers. **In F05, this modified data is NOT propagated back into the originating operation's params** (per N1) — only block semantics are honoured. Documented in code comments + the close-out evidence file.

3.4 Resolution order

Layer order (earliest layer wins on identical id): 1. Project file (<repoRoot>/ .helixcode/ hooks.yaml) 2. User file (~/ .helixcode/hooks.yaml) 3. Anything programmatically registered by HelixCode itself (built-ins; F05 ships none — N6)

Disabled hooks are loaded but not registered with the manager.

4. Wiring points

4.1 internal/tools/registry.Execute

```
Execute(ctx, name, params):
  event_before := NewEventWithContext(ctx, HookTypeBeforeToolCall)
  event_before.Source = "tool_registry"
  event_before.SetData("toolName", name)
  event_before.SetData("params", params)
  results := hooksMgr.TriggerEventAndWait(event_before)
  if blockers := hooks.Blockers(results); len(blockers) > 0:
    return nil, wrap_blockers(blockers)

  if name == "Bash":
    # fire HookTypeBeforeBash with same payload, same blocker logic
```

```

elif name in {Edit, Write, MultiEdit}:
    # fire HookTypeBeforeEdit similarly

result, execErr := tool.Execute(ctx, params)

# Always fire after-events (success OR error) for observability.
event_after := NewEventWithContext(ctx, HookTypeAfterToolCall)
event_after.Source = "tool_registry"
event_after.SetData("toolName", name)
event_after.SetData("params", params)
event_after.SetData("result", result)
event_after.SetData("error", execErr)
afterResults := hooksMgr.TriggerEventAndWait(event_after)
# blockers on after-events: logged at WARN, not propagated (op already

if name == "Bash":
    # fire HookTypeAfterBash similarly
elif name in {Edit, Write, MultiEdit}:
    # fire HookTypeAfterEdit similarly

return result, execErr

```

4.2 internal/llm/compression/auto_compactor.go

After F01's AutoCompactor produces a compacted message window, dispatch OnCompaction synchronously. A block here returns an error from the compaction; the next message-loop iteration handles the error.

4.3 internal/agent/agent.go

- On any tool error or LLM error in the message loop, fire OnError async (fire-and-forget).
- Add RequestPlanApproval(ctx context.Context, plan string) error method. It dispatches OnPlanApproval synchronously with payload {plan_text}, returns the aggregated blocker as an error (or nil if all hooks allow). **No production code calls this in F05**; F08 (Plan Mode) will wire it into the plan-approval gate.

5. CLI surface

helixcode hooks list	# tabwriter rows
helixcode hooks test <event-name>	# simulate event, print per-
helixcode hooks validate	# parse YAML, report errors
helixcode hooks enable <id>	# set enabled=true in user's
helixcode hooks disable <id>	# set enabled=false

Slash command:

/hooks	# = list
/hooks list	# explicit
/hooks test <event>	# simulate

/hooks aliased to /hk. Mutations (enable/disable) deliberately not exposed via slash command — they touch the user's filesystem and should go through Cobra (consistent with F02 / permissions not exposing add/remove mutations to the slash command).

6. Error handling

Case	Behaviour
Malformed YAML	Load returns error wrapping yaml's line:col; CLI startup fails fast
Missing or wrong apiVersion	Load returns error: "unsupported apiVersion %q (expected helixcode.hooks/v1)"
Unknown event type in YAML	hook entry is rejected; logged at WARN; other entries continue to register
Script not found at fire time	hook returns error wrapping os.ErrNotExist; treated as block (fail-closed)
Script timeout	context.DeadlineExceeded returned; treated as block
Async hook error	logged at WARN; does NOT affect originating operation
Blockers finds blockers on a before-event	caller wraps them: "operation blocked by N hook(s): <hook-id>: <stderr>"; aborts
Blockers finds blockers on an after-event	logged at WARN; not propagated (op already happened)
Modify-payload JSON malformed	logged at WARN; original payload preserved; not treated as block
RequestPlanApproval blocker	wrapped error returned; F08's caller decides how to surface

7. Testing strategy (CONST-035 / Article XI §11.9)

7.1 Unit (internal/hooks/*_test.go)

- Hook-type enum: every new HookType constant has a string round-trip test.
- yaml_loader.go: malformed YAML, missing apiVersion, project-overrides-user identical id, disabled hooks loaded-not-registered, unknown event type rejected at load with WARN.
- shell_runner.go: real shell script `#!/bin/sh\nexit 0` → no error; non-zero exit → error wrapping stderr; timeout → context.DeadlineExceeded; missing script → fail-closed; stdout JSON modify-payload parsed into event.Data; malformed stdout JSON logged + ignored.
- blockers.go: nil results → nil; results with non-nil errors → returned in order; results with nil errors → not returned.

Mocks ALLOWED at unit layer per CLAUDE.md, but most tests use REAL bash against `t.TempDir()` scripts.

7.2 Integration (tests/integration/hooks/hooks_integration_test.go, -tags=integration)

- Real ephemeral shell scripts in `t.TempDir()`, real `tools.ToolRegistry + hooks.Manager`, real fake Tool that records its invocation count.
- Asserts: `before_tool_call` block prevents `Tool.Execute` (invocation count stays at 0).
- Asserts: `after_tool_call` fires even when `Tool.Execute` returns error (invocation count = 1, after-script ran).
- Asserts: async hook doesn't block synchronous critical path (measure dispatcher latency).

No mocks at this layer (per CLAUDE.md / Constitution Rule 5).

7.3 Challenge (tests/e2e/challenges/hooks/)

End-to-end via Go-built driver: - **S1 — block-bash-rm**: `hooks.yaml` registers a `before_bash` hook pointing to a shell script that exits 1 if the bash command contains `rm -rf`. Driver attempts a `Bash(rm -rf /tmp/marker)` call via the real registry. The marker file must be **preserved** (`Tool.Execute` never ran). - **S2 — audit-after-tool**: `after_tool_call` hook writes a JSONL line per call to a log file. Driver makes 3 distinct tool calls. Log must have exactly 3 lines. - **S3 — yaml-validate**: write a malformed `hooks.yaml` to a temp project. `helixcode hooks validate --config <path>` exits non-zero with a clear error pointing at the offending line.

Runtime evidence: per-scenario stdout + filesystem inspection (marker presence, log line count) pasted in close-out commit body.

7.4 Anti-bluff smoke

```
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder"
internal/hooks/ cmd/cli/hooks_cmd.go internal/commands/
hooks_command.go tests/integration/hooks/ tests/e2e/challenges/
hooks/ returns clean.
```

7.5 Mutation test (CONST-039)

Comment out `if blockers := hooks.Blockers(results); len(blockers) > 0 { return ...err... }` in `tools.registry.Execute`. Re-run Challenge: S1 MUST fail (`rm` runs because the hook block is no longer honoured). Revert and confirm PASS.

8. Sub-task plan

13 tasks (largest in the programme so far — wiring spans 4 packages):

#	Task	Outputs
T01	Bootstrap evidence + advance PROGRESS	docs only
T02	Add 6 new <code>HookType</code> constants to <code>internal/hooks/hook.go</code> (TDD round-trip tests)	unit tests pass

#	Task	Outputs
T03	<code>yaml_loader.go</code> — <code>FileLoader</code> with <code>apiVersion</code> validation + project-over-user override (TDD)	unit tests pass
T04	<code>shell_runner.go</code> — generic shell-runner <code>HookFunc</code> with timeout + missing-script handling (TDD)	unit tests pass
T05	<code>blockers.go</code> — <code>Blockers([]ExecutionResult) []error</code> helper (TDD)	unit tests pass
T06	Wire <code>BeforeToolCall/AfterToolCall</code> + specialised <code>BeforeEdit/AfterEdit/BeforeBash/AfterBash</code> into <code>internal/tools/registry.Execute</code> (TDD)	unit tests pass
T07	Wire <code>OnCompaction</code> into <code>internal/llm/compression/auto_compactor.go</code> (TDD)	unit tests pass
T08	Wire <code>OnError</code> into <code>internal/agent/agent.go</code> message loop + add <code>RequestPlanApproval(ctx, plan)</code> stub method dispatching <code>OnPlanApproval</code> (TDD)	unit tests pass
T09	<code>cmd/cli/hooks_cmd.go</code> Cobra helixcode hooks <code>{list, test, enable, disable, validate}</code> + dispatcher in <code>main.go</code>	unit + smoke
T10	<code>internal/commands/hooks_command.go</code> slash command + <code>RegisterBuiltinCommandsWithHooks</code> extension in <code>builtin/register.go</code> (TDD)	unit + builtin-registry test
T11	<code>cmd/cli/main.go</code> startup wiring: construct <code>FileLoader</code> , load YAML, register enabled hooks into <code>session.Manager.hooksManager</code> . Integration tests (no mocks, real ephemeral shell scripts)	- tags=integration test passes
T12	Challenge: 3 scenarios from §7.3 with runtime evidence	Challenge PASS
T13	Feature 5 close-out: anti-bluff scan, verify-foundation, push to all 4 remotes (no force)	PROGRESS flipped to F06

13 sub-commits expected. F06 (MCP Full Lifecycle) unblocked when T13 lands.

9. Out of scope for F05

Listed in §1.3 (N1–N7). Summarised: modify-payload propagation back to operation params, production caller for plan-approval, hook UI, payload schema validation, worktree-aware hook discovery, built-in hooks, YAML hot-reload.

10. Risks and mitigations

Risk	Mitigation
Existing hooks <code>.Manager</code> API may be insufficient (e.g., no way to wait for sync handlers in error path)	Audit at T02; <code>Manager.TriggerEventAndWait</code> already exists per project exploration. If gaps emerge, add minimal helpers; don't rebuild.
Wiring touches 4 packages — reviews may find pre-existing bluffs in adjacent code	Anti-bluff smoke is scoped to NEW files + the lines added by F05; pre-existing hits in <code>cmd/cli/main.go</code> are already documented as F02/F03 follow-ups, not F05's responsibility.
F08's <code>RequestPlanApproval</code> integration may need a different signature later	Document the signature in a <code>// F08 wires this</code> comment; F08's plan-approval task can refactor if needed. The stub fires a real event via the dispatcher, so even without F08, the event surface is testable via <code>helixcode hooks test on_plan_approval</code> .
Shell scripts may have different shebangs across hosts (bash vs sh vs python)	<code>shell_runner</code> uses <code>exec.CommandContext</code> with the script path directly — the kernel honours the script's own shebang. No assumption about which interpreter.
Script timeouts under load	Default timeout is 0 (no timeout), per existing Hook struct. Users opt in via YAML.
Async hook race with after-event ordering	F05 documents that async hooks fire-and-forget; their relative ordering vs after-events is undefined. Users wanting ordering use sync hooks.
<code>helixcode hooks enable/disable</code> mutating user's YAML may corrupt comments	Use <code>gopkg.in/yaml.v3</code> Node-based round-trip for the mutation path; preserves comments. (T09 has this.)

11. References

- Synthesis spec: `docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md` §4.1 (Phase 1 charter)
- Porting doc: `docs/improvements/04_main_plan_step_02/kimi_agent_helix_cli_integration_blueprint/porting_claude_code.md` §Feature 5
- Predecessor spec: `docs/superpowers/specs/2026-05-05-p1-f04-git-worktree-agent-isolation-design.md` (commit 7ba8907)
- Predecessor plan: `docs/superpowers/plans/2026-05-05-p1-f04-git-worktree-agent-isolation.md` (commit 7abf0c7)
- Evidence file (live): `docs/improvements/06_phase_1_evidence.md`

- Existing infrastructure being extended (NOT replaced):
 - `helix_code/internal/hooks/` — existing Manager + Hook + Event + Executor; F05 adds 6 HookType consts + 3 new files (`yaml_loader.go`, `shell_runner.go`, `blockers.go`)
 - `helix_code/internal/session/manager.go` — already has `hooksManager *hooks.Manager` field; F05 reuses it
 - `helix_code/internal/tools/registry.go` — extended with 6 dispatch points in `Execute`
 - `helix_code/internal/llm/compression/auto_compactor.go` — extended with `OnCompaction` dispatch
 - `helix_code/internal/agent/agent.go` — extended with `OnError` dispatch + `RequestPlanApproval` stub
 - `helix_code/internal/commands/builtin/register.go` — extended with `RegisterBuiltinCommandsWithHooks`
- Constitutional anchors:
 - Article XI §11.9 — Anti-Bluff Forensic Anchor
 - CONST-035 — Zero-Bluff Mandate
 - CONST-039 — Challenge System Integrity (mutation testing mandatory)
 - CONST-042 — No-Secret-Leak (script paths in YAML may reveal local paths but no credentials)
 - CONST-043 — No-Force-Push (close-out commit T13 pushes without force)

End of P1-F05 Hook-Based Extensibility design spec.